

Group 11 - Chess AI Report

Introduction

Chess is among the most popular board games due to its depth, complexity, and elegance, with diverse play styles and strategies. In fact, there are as many as 10^{120} possible games of chess that can be played - a number that exceeds even the number of atoms in the observable **universe** (10^{80}). Our primary motivation for building this project was to determine whether we could develop a chess engine capable of competing with the world's best chess players.

Background

Stockfish is the strongest chess engine on the planet (currently). The average game of chess lasts approximately 40 moves, but the time spent per move is too variable to provide a reliable estimate. However, assuming unlimited time, a human player would take approximately 2-3 minutes per move in an average position and closer to 10 minutes in a critical position.

Methodology

We split our group into three subgroups: two developing AI models using Game Playing Algorithms (Minimax and MCTS), and a third developing an ANN to predict the evaluation score for a position. This structure was designed to compare the models to determine which would be the strongest.

Minimax

Evaluation:

- a) **Feature Selection:** The evaluation function consists of nine key features: Piece-Square Tables, King Safety, Mobility, Material, Bishop Pair, Open Rook Files, Center Control, and Knight Outposts. Thirty features were initially considered, but due to diminishing returns and potential overfitting, we limited our selection to the nine features mentioned above. Furthermore, our choice of features and their number were guided by the observation that more features would increase move time, thereby limiting the depth of the Minimax algorithm.
- b) **Optimization:** Despite reducing the number of features, the evaluation remained slower than desired. To improve performance, we switched to bitboards, which represent the chessboard as a 64-bit integer, where '1' signifies a piece and '0' indicates an empty square. This approach enables us to access any square in constant time, $O(1)$.
- c) **Score Handling:** The evaluation function computes a net score by summing the feature scores, each weighted by its weight. A positive score indicates an advantage for White, while a negative score indicates an advantage for Black. This method is independent of the moving side and follows the traditional approach used by most modern engines.

Search & Optimization

- a) Our search engine uses a Minimax algorithm with Alpha-Beta pruning, enhanced by several optimizations. The key improvement is the Transposition Table, which caches evaluations of previously analyzed positions. To boost Alpha-Beta pruning efficiency, we also utilize Iterative Deepening and strong move ordering techniques, like the history heuristic and killer moves, which prioritize high-value moves and increase cutoffs effectively.
- b) We enhanced evaluation stability through Quiescence Search, which prioritizes captures and forcing moves over noisy leaf positions. Static Exchange Evaluation effectively filters out losing captures, while Null Move Pruning allows us to bypass quiet subtrees, reducing the search tree without compromising accuracy.

- c) The optimizations greatly improved the engine's speed, depth, and reliability over a standard Minimax approach. We tested the Minimax algorithm at depth 5 by playing 50 games against a Stockfish engine rated 2000 Elo, with a 2-second time limit per move, and analyzed each Minimax move with a second Stockfish engine.

Results

- a) **As White:** In a match where Minimax played as White against a 2000-Elo Stockfish with a 2-second move limit, the results were closely contested: Minimax won 24 games, lost 25, and drew once as evident in Figure 1. The average move time was about 13 seconds, resulting in games lasting around 16 minutes. The nodes searched versus time in Figure 6 revealed a linear pattern, indicating efficient search optimizations. Minimax achieved a 44% transposition table hit rate and utilized SEE pruning to eliminate approximately 11% of tactical branches, effectively reducing the search space. An analysis in Figure 3 found that 83% of Minimax's moves were rated as "OK" compared to Stockfish's deeper analysis, with few inaccuracies, mistakes, and blunders. The centipawn loss histogram 2 peaked near zero, showing most moves were close to optimal.
- b) **As Black:** When our Minimax engine played as Black against Stockfish, its performance declined compared to its games as White. Out of 50 matches, Minimax won 17, lost 32, and drew 1, as evident in Figure 7. Despite the lower win rate, internal performance metrics were comparable to those in White-side games, with an average computation time of about 13 seconds per move. The linear relationship between nodes searched and time, Figure 10, indicated stable search efficiency, and the transposition table hit rate was around 40-45%. In terms of move quality, approximately 80% of Minimax's moves were classified as "OK" compared to Stockfish's best choices, while about 12% were inaccuracies (Figure 9). The centipawn loss histogram (Figure 12) showed most moves clustered near zero, with higher losses occurring in already losing positions. This indicates that Minimax performs well in close games but struggles when behind. The move-agreement in Figure 11 showed an agreement rate of approximately 40%, comparable to its White-side performance. However, Stockfish was more effective at converting Minimax's inaccuracies into decisive advantages, contributing to a significant win-rate disparity while maintaining comparable search and pruning efficiency.

ANN Evaluation Function

Data Generation

- a) **Simulation:** The goal was to develop an Artificial Neural Network (ANN) model to predict chess position evaluations and integrate it with gameplay algorithms like Alpha-Beta Minimax and MCTS. A dataset of 225,000 positions was created by simulating 2,000 games across various skill levels with the Stockfish Chess engine. In total, 40 bots were generated, with four representing each Stockfish skill level range in increments of 2 (e.g., 1-2, up to 19-20), using randomized thread counts(1-4) and hash sizes (16-64mb) to ensure diverse positions.
- b) **Early fallacy:** In the early stages, the game length was limited to 70 positions per player to mimic human play, which averages around 40 positions. Games that exceeded this limit were labeled as draws, which caused issues in high-level bot vs. bot games and led to 15% of games being incorrectly recorded as draws.

Feature Engineering & EDA

- a) **Response:** Initially, the game outcome in three variants was designated as the response variable: win(2)/draw(1)/loss(0), scaled outcome adjusting for the skill of the opponent

(win = $1 * 1.5^{\text{skill}}$), and scaled with reward/penalty for tempo (up to $\pm 10\% * \text{win}$). All three variants have proven inadequate as they suggest that models can predict game outcomes from a single position. Therefore, we used Stockfish's evaluation value as the response variable.

- b) **Null Distribution:** Initially, 28 balance features ('white' - 'black') were extracted and later expanded to 76, including separate attributes for each player. This was done to improve model performance and reduce the high nullity in the balanced features, as shown in Table 1.
- c) **Correlation Analysis:** Feature-to-feature correlation analysis was performed (see Figure 13) to remove or adjust high-correlation feature pairs, such as the overlap between 'mobility' and 'safe_mobility.' Additionally, a feature-to-response correlation analysis was conducted (refer to Figure 14) to address low correlation values by either removing or aggregating them, such as by summing all 'piece_sqr_sum' features.

Methodology & Results

- a) **Early Modeling:** We used a Linear Regression model with a Ridge Penalty for its efficiency. We observed low performance across feature subsets, particularly due to encoding checkmate positions as ± 9999 , which adversely affected results. As a result, we excluded these positions and those with minimal advantage for either player (ranging from -10 to 10) from our ANN modeling.
- b) **Hyperparameters:** Using a curated dataset of 124,000 positions and standardized 34 features, grid search was employed to identify the optimal ANN hyperparameters. Each model was trained for 20 epochs with 3-fold cross-validation and Early Stopping to prevent overfitting. The resulting subset of hyperparameters was (Table 2): hidden-layer neurons (1:512, 2:256, 3:128), learning rate (0.001), dropout rate (0.1), and activation function (ReLU).
- c) **Results:** The final model achieved the following performance on the unseen test subset: R^2 (0.8971), RMSE (16.52), and MSE (27.29). This model and feature extraction were integrated and optimized, increasing predictions per second from 10 to 228. In gameplay with the Minimax algorithm, it took 0.5s (d=2), 4s (d=3), and >1min(d=4) per move.

Monte Carlo Tree Search

Methodology

- a) **Implementation:** The Monte Carlo Tree Search (MCTS) algorithm manages board states by encapsulating them within a Node class. This class captures not only the current board state but also the previous state and any immediate future states. When it's time for the bot to make a move, it systematically follows a 4-phase plan to identify the optimal move.
- b) **Selection & Expansion:** During the selection and expansion phase, an unsimulated chess move is chosen for simulation. If all possible moves from the current board state have already been simulated, the algorithm will select the move with the highest upper confidence bound value for simulation. This action refreshes the list of unsimulated chess moves available for selection.
- c) **Simulation & Backpropagation:** In the simulation phase, random chess moves are simulated until the game ends or the maximum depth is reached. After the simulation, the board state is evaluated for backpropagation, where the selected move and its preceding moves receive the evaluation value.

Optimizations

- a) **Remember:** To improve the MCTS bot's efficiency and speed, the 'Remember' feature was added. This addresses the issue of the bot not recording the scores obtained during backpropagation in previous moves.
- b) **Look at Tried Moves:** During the selection and expansion phase, only untested moves are considered in later stages of the game. To address this, the 'Look at Tried Moves' feature was adopted to counter early biases from random playouts.
- c) **Jump the Win Gun:** When the MCTS bot finds a path to a winning condition, it keeps simulating games. 'Jump the Win Gun' feature was developed, stopping the simulation upon discovery of a winning path.

Results

- a) **Methodology:** To gather statistics, we had the program compete against itself and Stockfish at two distinct ELO ratings: 1320 and 1900. To reduce data-collection time, we set a limit of 300 total moves per game. This limit was only reached during matches between the program and a copy of itself, where both bots would "dilly-dally," effectively stalling the game.
- b) **Game Duration:** When models played against themselves, there was a larger number of moves than when played against the Stockfish engine, where games ended faster, as evident in Figure 15. To extend this finding, the average game length when playing against Stockfish (Figure 16) shows an equal trend with average time per move (Figure 17)
- c) **Simulations:** To assess the effect of optimizations on the evaluation score, we examined Figures 18-20. Reducing the number of simulations does not consistently affect the evaluation score of the resulting board state. While some optimizations lead to improvements, others do not. Significantly, none of the optimizations raised the evaluation score above negative values, except for the "Look at Tried Moves" strategy.
- d) **Win Rate:** Upon examining Figures 21 and 22, we found that the "Look at Tried Moves" feature increases the number of simulated moves considered, including winning sets. However, despite identifying many winning moves, this did not result in actual game wins, likely due to the randomness in move selection during simulation.

Discussion

Each approach revealed distinct trade-offs. The Minimax engine showed near-master-level tactical strength against 2000 Elo Stockfish but exhibited slower move times and a noticeable performance drop when playing as Black. The Monte Carlo Tree Search excelled in speed and scalability, yet suffered from tactical inaccuracies due to random simulations. The Artificial Neural Network delivered the most accurate static evaluation but was constrained by feature-extraction overhead. These results highlight that no single method dominates, and future work should explore hybrid systems.

Conclusion:

We successfully developed and compared three chess AI approaches. The Minimax-based engine proved competitively strong, the MCTS implementation achieved excellent speed, and the neural network delivered the most accurate position evaluation. While each method has clear strengths and limitations, this project demonstrates that modern chess engines are most powerful when hybridizing classical and learning-based techniques.

Figures

Minimax

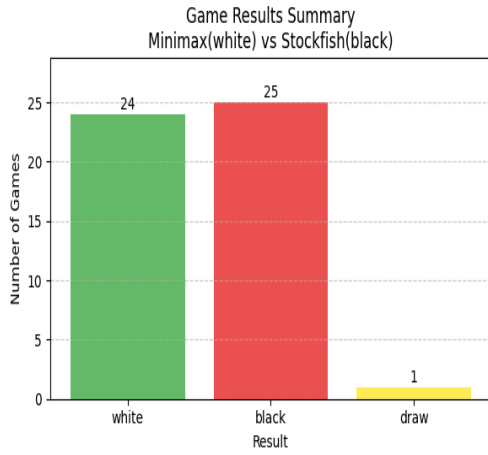


Figure 1. (White) Game results for Minimax vs. Stockfish (Black)

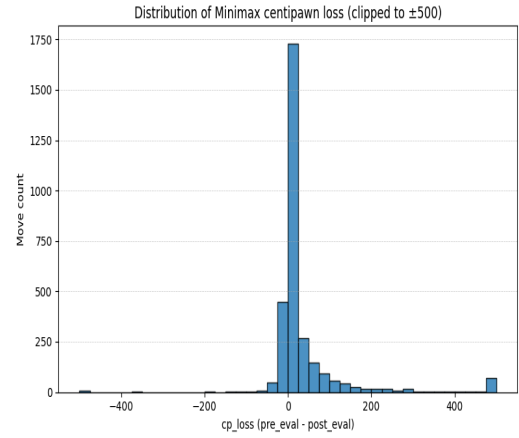


Figure 2. (White) Distribution of Minimax centipawn loss

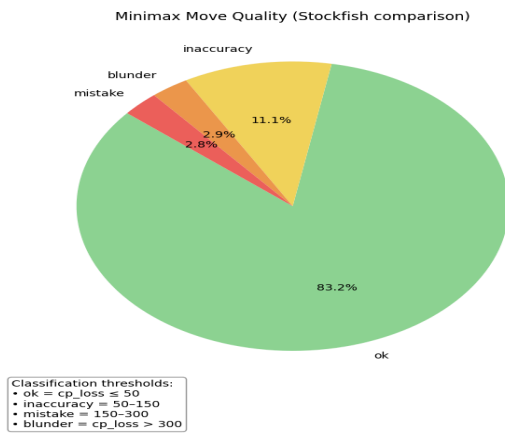


Figure 3. (White) Minimax move-quality breakdown compared to Stockfish.

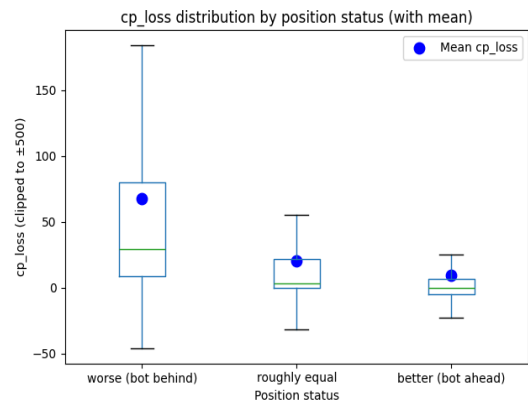


Figure 4. (White) Centipawn-loss distribution across worse, equal, and better positions.

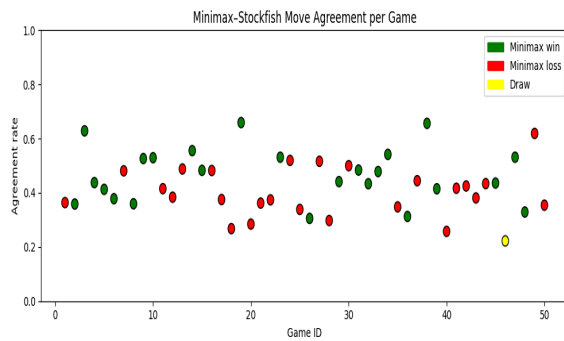


Figure 5. (White) Per-game move agreement rate between Minimax and Stockfish.

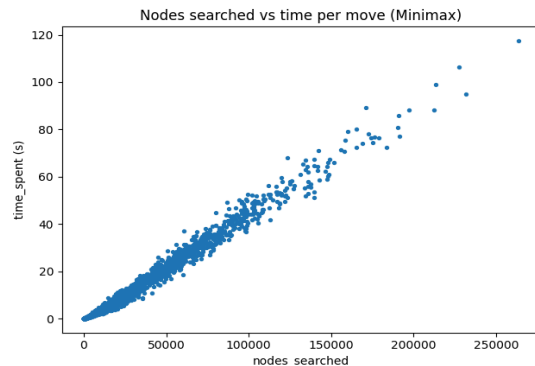


Figure 6. (White) Relationship between nodes searched and time spent per move.

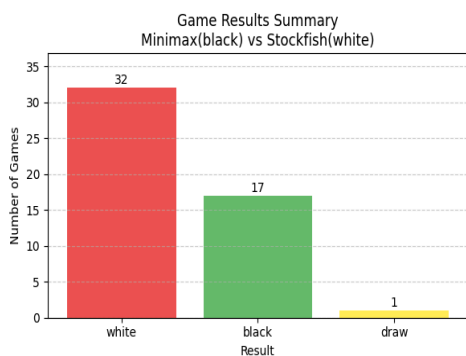


Figure 7. (Black) Game results for Minimax (Black) vs. Stockfish (White)

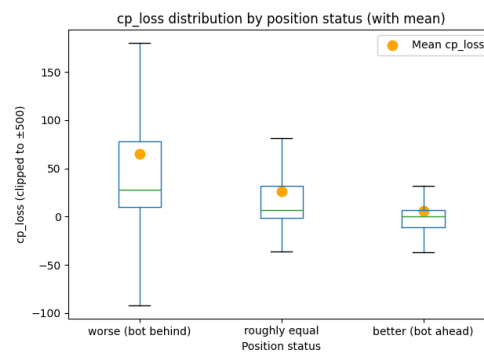


Figure 8. (Black) Centipawn-loss distribution across worse, equal, and better positions.

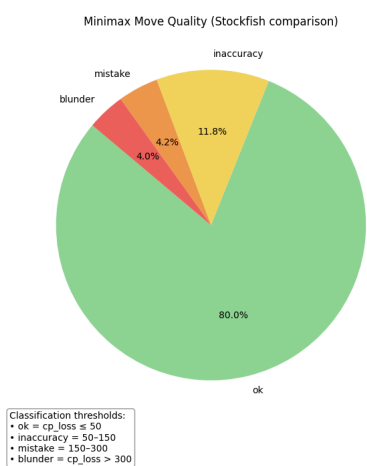


Figure 9. (Black) Minimax move-quality breakdown compared to Stockfish.

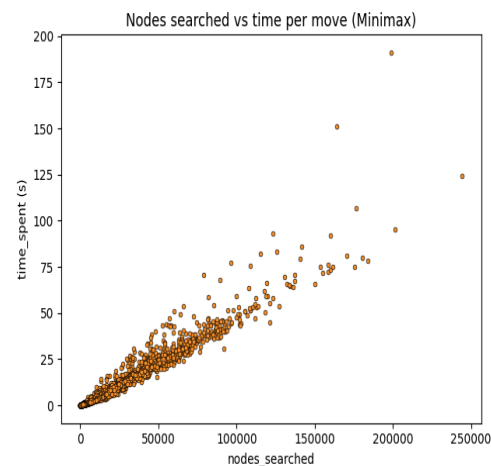


Figure 10. (Black) Relationship between nodes searched and time spent per move.

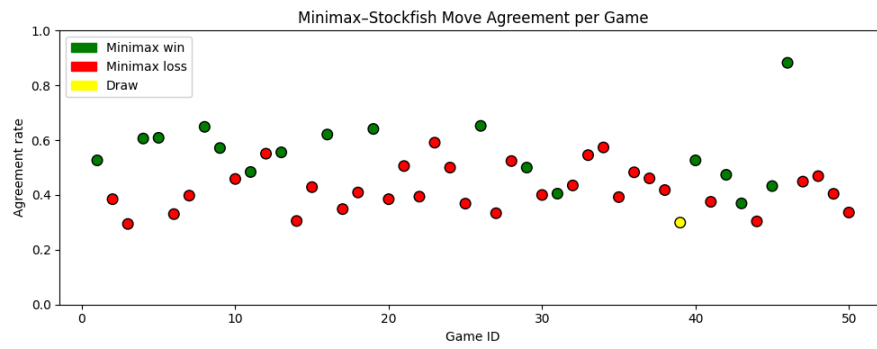


Figure 11. (Black) Per-game move agreement rate between Minimax and Stockfish.

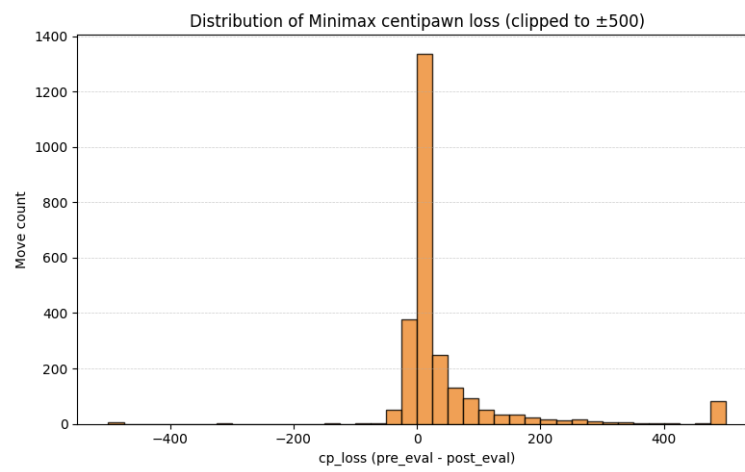


Figure 12. (Black) Distribution of Minimax centipawn loss.

ANN Evaluation Function

Feature	0-20	20-40	40-60	60+	Overall
attack_balance	8.7%	3.7%	4.0%	7.2%	6.1%
attack_black	0.0%	0.0%	0.0%	0.0%	0.0%
attack_white	0.0%	0.0%	0.0%	0.0%	0.0%
bishop_outposts_black	98.9%	96.7%	96.2%	98.0%	97.6%
bishop_outposts_white	98.6%	97.0%	97.6%	98.6%	98.0%
bishop_pair_black	20.5%	79.5%	95.9%	99.2%	64.7%
bishop_pair_white	21.9%	80.7%	96.6%	98.9%	65.7%
bishop_sqr_sum	39.1%	35.2%	55.0%	67.4%	45.6%
bishop_sqr_sum_black	18.8%	51.6%	69.4%	78.7%	47.7%
bishop_sqr_sum_white	20.3%	52.4%	73.2%	80.7%	49.4%
center_attackers_black	5.8%	3.5%	13.3%	26.6%	10.0%
center_attackers_white	3.0%	3.6%	14.2%	28.4%	9.5%
connected_pawns	34.6%	27.1%	45.0%	81.3%	42.3%
connected_pawns_black	0.4%	14.1%	49.6%	85.6%	27.7%
connected_pawns_white	0.4%	13.3%	46.8%	85.9%	27.0%
defense_balance	12.5%	9.7%	13.5%	28.0%	14.5%
defense_black	10.4%	5.5%	23.2%	55.5%	19.0%
defense_white	10.5%	5.7%	22.8%	53.1%	18.6%
doubled_pawns	77.3%	67.1%	76.8%	93.4%	77.0%
doubled_pawns_black	85.7%	79.5%	86.3%	96.3%	85.8%
doubled_pawns_white	88.1%	80.7%	87.5%	96.7%	87.3%
half_open_king_files	65.1%	42.0%	32.3%	41.7%	48.5%
half_open_king_files_black	66.0%	41.3%	16.6%	3.4%	39.3%
half_open_king_files_white	66.6%	43.7%	16.6%	3.2%	40.2%
isolated_pawns	73.5%	31.2%	28.7%	42.0%	47.7%
isolated_pawns_black	81.9%	36.5%	26.0%	51.2%	53.4%
isolated_pawns_white	83.7%	40.6%	26.7%	47.6%	54.8%
king_ring_enemy_pressure	58.8%	18.6%	9.6%	16.3%	31.0%
king_ring_enemy_pressure_black	69.6%	36.5%	25.1%	31.2%	45.4%
king_ring_enemy_pressure_white	75.5%	39.2%	24.2%	33.1%	48.5%

knight_sqr_sum_black	11.1%	54.2%	75.9%	86.7%	48.2%
knight_sqr_sum_white	9.9%	52.3%	76.9%	87.7%	47.6%
material_bishop_black	2.3%	30.1%	58.9%	72.7%	32.5%
material_bishop_white	2.1%	31.6%	63.4%	74.8%	34.0%
material_knight_black	4.7%	42.7%	70.9%	84.6%	41.3%
material_knight_white	4.5%	42.4%	72.0%	85.4%	41.5%
material_pawn_black	0.0%	0.0%	3.3%	39.9%	7.5%
material_pawn_white	0.0%	0.0%	2.6%	38.0%	7.1%
material_queen_black	8.9%	46.3%	74.9%	83.5%	44.3%
material_queen_white	8.5%	45.7%	74.6%	83.7%	44.0%
material_rook_black	0.1%	9.4%	36.2%	54.1%	18.4%
material_rook_white	0.0%	9.7%	36.4%	52.4%	18.2%
mobility_balance	13.3%	8.1%	10.5%	20.0%	12.4%
mobility_black	3.3%	3.1%	10.0%	22.7%	7.8%
mobility_safe_balance	7.4%	3.7%	3.8%	5.8%	5.4%
mobility_safe_black	0.1%	0.3%	0.4%	0.2%	0.2%
mobility_safe_white	0.1%	0.2%	0.5%	0.2%	0.2%
mobility_white	3.6%	2.9%	10.9%	23.3%	8.1%
outposts_black	95.4%	91.9%	92.9%	97.4%	94.3%
outposts_white	94.5%	91.1%	94.3%	97.6%	94.0%
passed_pawns	92.0%	51.2%	30.4%	39.0%	60.1%
passed_pawns_black	95.5%	66.7%	46.2%	59.1%	72.2%
passed_pawns_white	95.7%	66.4%	44.4%	58.2%	71.7%
pawn_shield	24.5%	19.8%	35.0%	67.8%	32.4%
pawn_shield_black	2.3%	15.8%	45.9%	80.1%	27.3%
pawn_shield_white	3.1%	14.8%	47.6%	79.4%	27.5%
pawn_sqr_sum	9.4%	2.7%	4.4%	32.3%	10.5%
pawn_sqr_sum_black	0.0%	0.1%	6.5%	47.8%	9.5%
pawn_sqr_sum_white	0.0%	0.2%	6.1%	45.8%	9.1%
pieces_occupying_center	32.3%	31.2%	42.7%	57.1%	38.1%
queen_sqr_sum	62.4%	56.4%	74.3%	80.1%	65.7%
queen_sqr_sum_black	27.6%	68.0%	82.4%	88.1%	59.6%
queen_sqr_sum_white	28.0%	66.5%	81.9%	88.5%	59.3%
rook_sqr_sum	95.6%	68.6%	68.6%	74.7%	79.2%
rook_sqr_sum_black	97.9%	81.5%	80.6%	85.6%	87.9%
rook_sqr_sum_white	97.5%	81.4%	81.6%	85.0%	87.8%
sqr_sum_black	0.1%	1.2%	3.8%	3.5%	1.7%
sqr_sum_white	0.0%	1.1%	4.0%	3.7%	1.7%
target	0.5%	0.1%	0.2%	0.5%	0.3%
threat_balance	32.4%	20.8%	32.1%	53.8%	32.6%
threat_black	10.5%	5.8%	22.8%	54.2%	18.8%
threat_white	10.4%	5.4%	23.2%	54.7%	18.8%

Table 1. Distribution of positions with zero value per feature in position range increments

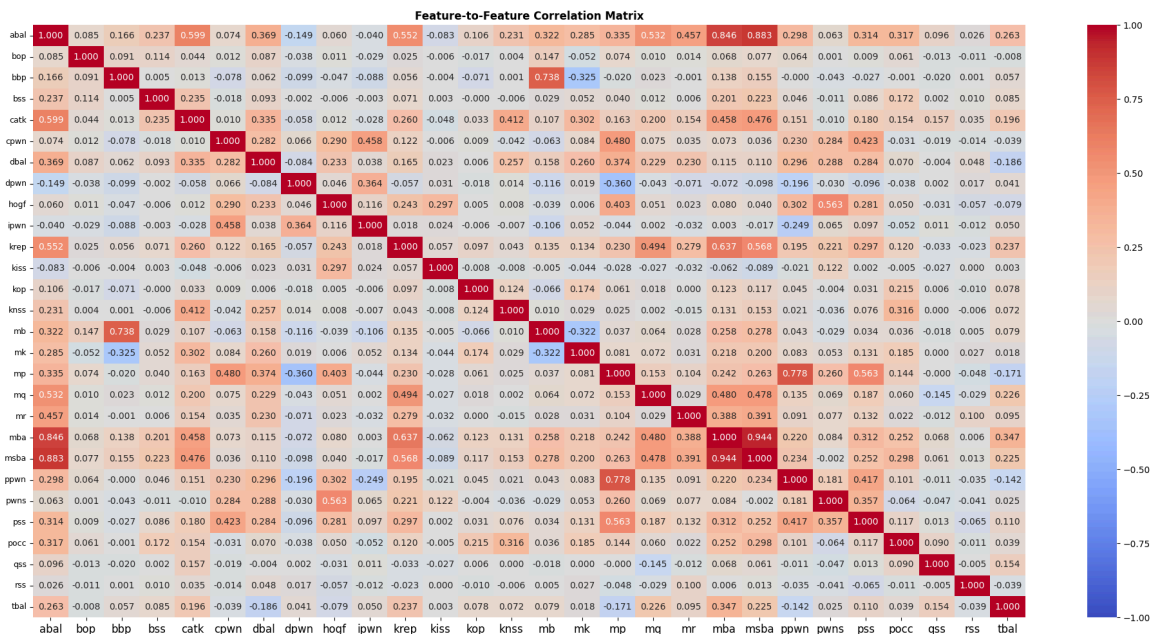


Figure 13. Feature to Feature Correlation Matrix

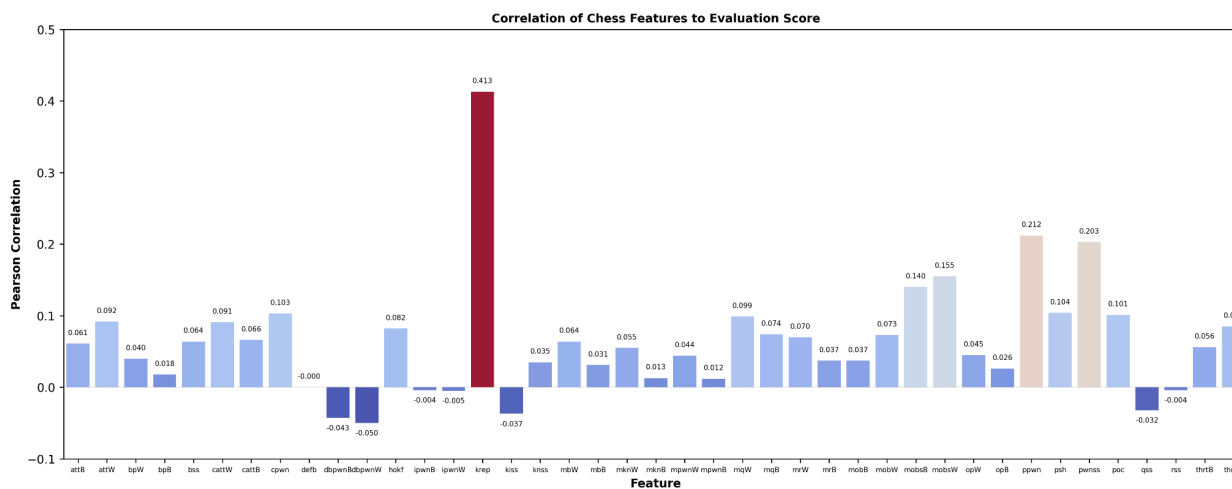


Figure 14. Feature to Evaluation Score Correlation

Neurons	Dropout	LR	Activation	MSE	RMSE	R²
512	0.1	0.001	relu	3.020	1.738	0.8854
512	0.2	0.001	relu	3.054	1.748	0.8841
512	0.1	0.001	tanh	3.069	1.752	0.8836
512	0.2	0.001	tanh	3.081	1.755	0.8831
256	0.1	0.001	relu	3.089	1.758	0.8828
256	0.2	0.001	tanh	3.238	1.799	0.8772
256	0.1	0.001	tanh	3.196	1.788	0.8787
256	0.1	0.01	relu	3.278	1.811	0.8756
512	0.1	0.01	tanh	3.395	1.843	0.8712
256	0.1	0.01	tanh	3.383	1.839	0.8717
512	0.2	0.01	tanh	3.446	1.856	0.8693
256	0.2	0.01	tanh	3.438	1.854	0.8696
512	0.2	0.01	relu	3.746	1.935	0.8579
256	0.2	0.01	relu	3.845	1.961	0.8541

Table 2. Grid Search Parameters Results

Monte Carlo Tree Search (MCTS)

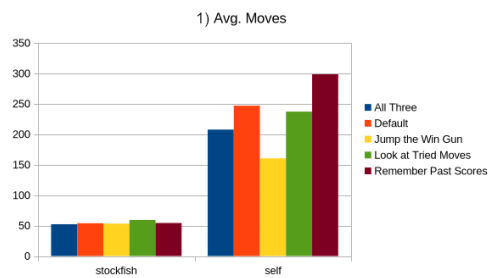


Figure 15. The average number of moves performed by the MCTS bot and Stockfish.

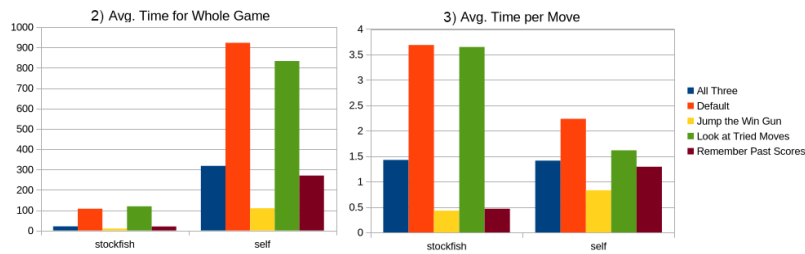


Figure 16. (left) The average time it took for a chess game to complete
 Figure 17. (right) The average time spent thinking about what move to play.

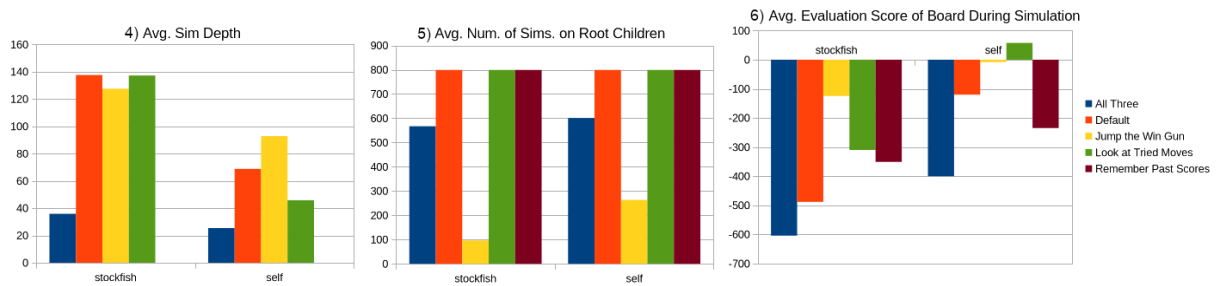


Figure 18. (left) The average depth reached during the simulation phase
 Figure 19. (middle) The average number of simulations performed on immediate moves from the current state
 Figure 20. (right) The average chessboard evaluation score

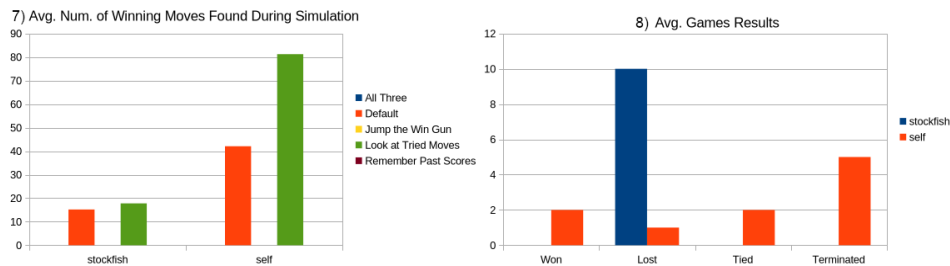


Figure 21. (left) The average number of winning moves found during simulation playing against itself and Stockfish.
 Figure 22. (right) The average number of games the MCTS bot won, lost, tied, and stalled against itself and Stockfish.

Contribution:

Yash Mago - Designed the evaluation function for Minimax bot, optimized the move ordering in search to enhance opening play, helped format and build the presentation, testing

Dhruv Kishnani - Added(to Minimax): Move ordering, Quiescence Search, Static Exchange Evaluation, Iterative Deepening, Aspiration Windows; testing, and collecting data

Tianrun Xu – Implemented Minimax/Negamax with alpha-beta pruning, designed and integrated the transposition table, added null-move pruning, and optimized search performance, testing

Charlie Edwards - Development of ANN evaluation function, feature selection and extraction correlation analysis, data generation

Sasa Antunovic - Development of ANN evaluation function, features selection and extraction correlation analysis, data generation, initial GUI implementation, integration with Minimax

Sween Zhou - Development of ANN evaluation function, features selection and extraction correlation analysis, data generation

Ayden Leu - Development of MCTS bot, code cleanup, bot optimizations, statistic gathering

Dehui Chen - Development of MCTS bot, bot data, debugging the evaluation function

Yuxuan Sun - Development of MCTS bot, implemented the initial code